Probabilistic Regression Identifiability — Research Notes

ClassReg vs ProbReg, the head-index swap degeneracy, and three candidate fixes

Apr 2026 session

2026-04-21

1. Executive summary

This document is the durable record of our investigation into the **head-index swap degeneracy** in ProbReg's `SEPARATE_HEADS` parametrization and the three orthogonal fixes we tested: an MDN-style likelihood, classifier–head gradient detachment (`CE_STOP_GRAD`), and anchored heads. It covers the theoretical motivation, the full 8-cell experimental matrix, the raw results across four toy datasets, visual interpretations of the probability curves, the recommendation we would take forward, a negative-result side experiment (the symlog hypothesis for heavy-tail targets), open questions, and a log of bugs found during this work. The goal is that a future session can resume without re-deriving the context.

Headline finding. Anchored heads were rejected on statistical grounds (§13) and replaced with an **ordering-constraint penalty** (§3.3) that imposes the minimal $k-1$ inequalities $M_0 < M_1 < ... < M_{k-1}$ on probability-weighted top-decile head means. The rerun (§9.1) shows the ordering penalty **works as intended, but only for Gaussian-LTV + REGRESSION_ONLY**: 2–7× anchor-error reduction with neutral-to-improved MSE/NLL on 7/8 cells. Under CE_STOP_GRAD the classifier already pre-orders the heads so the penalty is redundant; under MDN the head-means ordering has the wrong semantics and harms exponential/bimodal fits. CE_STOP_GRAD and MDN remain situational dials. ClassReg is consistently beaten by ProbReg except on narrow-range symmetric targets. One outstanding negative: on heavy-tail targets (exponential), CE_STOP_GRAD cells fail by an order of magnitude and **target-range compression via `symlog` does not rescue them** — the root cause is something other than target scale.

2. Background: architecture and the identifiability problem

2.1 ProbReg SEP_HEADS — what the model looks like

For a regression target $y \in \mathbb{R}$ and input $x \in \mathbb{R}^d$, ProbReg in the `SEPARATE_HEADS` configuration binarises y into k percentile-based bins and learns:

- A classifier $q_\phi(x) = \text{softmax}(\text{NN}_\phi(x)) \in \Delta^{k-1}$ producing class probabilities $p_i(x)$ for $i = 1, \dots, k$.
- k regression heads $h_i(p_i; \psi_i) \in \mathbb{R}^2$, each returning a per-class mean μ_i and log-variance $\log \sigma_i^2$.

The model's predictive mean and variance are combined via the **law of total variance** (LTV):

$$\mu_{\text{total}}(x) \;=\; \sum_{i=1}^{k} p_i(x)\,\mu_i(p_i(x)),$$

$$\sigma^2_{\text{total}}(x) \;=\; \underbrace{\sum_{i=1}^{k} p_i(x)\,\sigma_i^2(p_i(x))}_{\text{within-class}} \;+\; \underbrace{\sum_{i=1}^{k} p_i(x)\,\mu_i^2(p_i(x)) - \mu^2_{\text{total}}(x)}_{\text{between-class}}.$$

The training loss is the Gaussian NLL with these two moments:

$$\mathcal{L}_{\text{LTV}}(y,x) \;=\; \tfrac{1}{2}\left(\log(2\pi\,\sigma^2_{\text{total}}) + \frac{(y-\mu_{\text{total}})^2}{\sigma^2_{\text{total}}}\right).$$

2.2 Why this is degenerate: head-index swap

The loss $\mathcal{L}_{\text{LTV}}$ depends only on $(\mu_{\text{total}}, \sigma^2_{\text{total}})$. For a fixed input x with $p_i(x) \to 1$, the total mean collapses to a single head output:

$$\lim_{p_i \to 1} \mu_{\text{total}}(x) \;=\; \mu_i(p_i = 1).$$

There is **no pressure in the loss** to make μ_i equal the centroid c_i of bin i (where $c_i = \mathbb{E}[y \mid y \in \text{bin}_i]$ from training data). Any consistent relabeling $\sigma \in S_k$ with $p'_i = p_{\sigma(i)}$, $\mu'_i = \mu_{\sigma(i)}$ yields the same loss. Classifier "probs" therefore do not have to match any specific bin — the mapping between head index and y-value is free.

2.3 Why ClassReg doesn't have this

In `ClassifierRegressionModel`, the classifier is supervised by cross-entropy against **hard bin labels** derived from y. The mapper $m_i \mapsto \widehat{y}$ (e.g. `LOOKUP_MEDIAN`) is data-driven, not learned via the regression loss. The hard labels pin head-index $\leftrightarrow$ bin-of-y exactly; there is no swap freedom. The cost is that the classifier must commit to a hard class structure, which hurts smooth regression.

3. Three orthogonal fixes (hypotheses)

3.1 MDN NLL

Replace the LTV Gaussian NLL with a mixture-density-network likelihood where probabilities enter the likelihood structurally:

$$\mathcal{L}_{\text{MDN}}(y,x) \;=\; -\log \sum_{i=1}^{k} p_i(x)\,\mathcal{N}(y;\mu_i,\sigma_i^2).$$

Because each component's likelihood scales with its own p_i, a relabeling changes the per-sample log-likelihood — the loss is **not** invariant under head-index swaps. Identifiability comes from the likelihood structure itself.

Implementation note (`automl_package/utils/losses.py:mdn_nll`): the naive evaluation of $\log \sum_j p_j\,\mathcal{N}_j$ is numerically unstable (both factors can underflow). The code computes $\log p_j$ and $\log \mathcal{N}_j$ separately and combines them via `torch.logsumexp` with an ϵ-floor on p_j, which is the standard Bishop 1994 form.

3.2 CE_STOP_GRAD

Supervise the classifier directly by cross-entropy against bin labels, and `detach()` the classifier's probabilities before they flow into the regression heads:

$$\mathcal{L}_{\text{CE}}(x) \;=\; -\sum_{i=1}^{k} \mathbb{1}[y \in \text{bin}_i] \, \log p_i(x), \qquad \tilde{p}_i = p_i.\text{detach}().$$

Total loss $\mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{LTV}}(\tilde{p}, \mu, \sigma)$.

Although the total loss is arithmetically a sum, the stop-gradient severs the gradient path: the classifier receives gradient only from $\mathcal{L}_{\text{CE}}$ (because $\tilde{p}$ is detached before the regression module), and the heads receive gradient only from $\mathcal{L}_{\text{LTV}}$ (because they do not appear in $\mathcal{L}_{\text{CE}}$). In `SEPARATE_HEADS` the classifier and heads share no parameters, so the two losses optimize disjoint parameter sets: the training dynamic is effectively two uncoupled optimizations sharing a forward pass. The classifier is pinned by hard labels (like ClassReg), and the heads fit the regression loss treating $\tilde{p}$ as a frozen input signal. Identifiability comes from hard bin supervision. This lack of co-adaptation is also the leading suspect for the exponential failure (§8.5).

3.3 Anchored heads

Reparametrise each head so its output **at $p_i = 1$ is structurally the bin centroid**:

$$h_i(p_i) \;=\; c_i \;+\; (1 - p_i)\, f_i(p_i),$$

where c_i is the precomputed mean of y in bin i and f_i is a small MLP. At $p_i = 1$, $h_i = c_i$ exactly, with no gradient path for deviation. For $p_i < 1$, the residual f_i retains full expressivity. Identifiability comes from a hard structural prior rather than loss structure.

The centroid target is wrong for edge bins — a real limitation. The head returns the per-class mean μ_i and the model combines them by LTV: $\mu_{\text{total}}(x) = \sum_j p_j(x)\, \mu_j(p_j(x))$. When $p_i(x) \to 1$, $\mu_{\text{total}}(x) \to h_i(1)$. Under MSE loss the optimal target for $h_i(1)$ is

$$h_i^\star(1) \;=\; \mathbb{E}\big[y \,\big|\, p_i(x) = 1\big].$$

This is conditioned on the **classifier's confidence**, not on bin membership. It is *not* equal to $c_i = \mathbb{E}[y \mid y \in \text{bin}_i]$ in general — $\{x : p_i(x) = 1\}$ is typically a strict subset of $\{x : y \in \text{bin}_i\}$, specifically the "deep interior" away from bin boundaries.

For edge bins the two expectations differ substantially:

- **Upper bin** ($i = k - 1$): samples with $p_i(x) = 1$ live in the deep upper interior close to $y_{\max}$. So $\mathbb{E}[y \mid p_{k-1} = 1] > c_{k-1}$. The correct anchor is higher than the centroid.
- **Lower bin** ($i = 0$): symmetric. $\mathbb{E}[y \mid p_0 = 1] < c_0$.
- **Inner bins**: bin distribution is (approximately) symmetric about c_i so $\mathbb{E}[y \mid p_i = 1] \approx c_i$ and the centroid anchor is fine.

Anchoring $h_i(1) = c_i$ therefore **systematically pulls edge-bin predictions toward the interior** — downward on the upper tail, upward on the lower tail. On heavy-tailed targets this is harmful; the data confirms it (Table in §5, exponential $k = 5$: anchored Cell B MSE = 1.84 vs unanchored Cell A MSE = 0.50).

For contrast, ClassReg with `LOOKUP_MEDIAN` does not have this problem: its mapper is built empirically from observed (p, y) pairs, effectively estimating $\mathbb{E}[y \mid p_i(x)]$ (or the median thereof) directly from training data rather than from the centroid.

Interaction with monotonic constraints — worse still. The `use_monotonic_constraints=True` option replaces the head's Linear layer with `monotonic_linear`, which uses `softplus` weights so that h_i is monotone non-decreasing (negated for `Monotonicity.NEGATIVE`). With anchor active:

- Upper class (monotone positive): maximum at $p_i = 1$, pinned to c_i, so $h_i(p_i) \le c_i$ for all p_i. The model cannot predict above c_{k-1} anywhere.
- Lower class (monotone negative): $h_0(p_0) \ge c_0$ always.

The model's output range is bounded inside $[c_0, c_{k-1}]$ — the tails of y outside this interval are structurally unreachable. This is a hard architectural limit for unbounded/heavy-tail targets. The identifiability sweep disabled `use_monotonic_constraints` so this combination was not stressed, but it is an important caveat for any downstream use.

Refinements to the anchor target. The centroid is the wrong target for edge bins. Several candidates were brainstormed in session; the selected proposal at the end is an *ordering constraint* — a soft loss that imposes only the relative ordering of head outputs at their high-confidence operating points, without fixing any specific values.

Discarded candidates (kept here for record):

- *Bin-extreme anchor for edges* ($y_{\max}/y_{\min}$): dominated by order-statistic noise; the anchor value changes under data resampling.
- *Empirical* $\hat{\mathbb{E}}[y \mid p_i \ge \tau]$ *at* $p = 1$: fixes the target but still anchors at an operating point the model never reaches.
- *Single-point operating-point anchor* $(p_i^\star, y_i^\star)$: requires a classifier warmup to measure $p_i^\star$; asymmetric — anchor one end but not the other.
- *Two-point operating-point anchor* $(p_i^{\text{hi}}, y_i^{\text{hi}})$ and $(p_i^{\text{lo}}, y_i^{\text{lo}})$: still requires a warmup to measure the probabilities; complicates strategies other than `CE_STOP_GRAD`.
- *Empirical-curve baseline* $\widehat{m}_i(p) = \widehat{\mathbb{E}}[y \mid p_i = p]$: most principled but requires a warmup and per-head regressor.
- *Hard anchor at* c_i *for middle bins only, drop for edges*: inconsistent treatment; middle bins still suffer the $p = 1$ extrapolation issue.
- *Range-parametrised head* $h_i(p) = y_i^{\text{lo}} + (y_i^{\text{hi}} - y_i^{\text{lo}})\,\sigma(g_i(p))$ with $y_i^{\text{hi}}, y_i^{\text{lo}}$ from in-bin y-quantile means: bounds the head's range but (a) the range is narrower than the bin itself when using quantile means (tail values become unreachable), and (b) hard range bounds are stronger than identifiability requires.

Selected candidate — ordering constraint at high-confidence operating points. The only property needed to break the head-swap degeneracy is that heads are *ordered* in their high-p outputs — not that they take specific values. Imposing an ordering is strictly weaker than any point-anchor or range-bound proposal, and is the minimal constraint sufficient to break the S_k permutation symmetry of the LTV loss.

Why $p \to 1$ is the natural reference. At low or moderate $p_i(x)$, the classifier is saying "this sample *might* be in bin i". We have no grounded constraint on what h_i should output there, because y could be in any of several bins. Only as $p_i \to 1$ does the classifier assert "y is in bin i with certainty", and then the law of iterated expectations ties the head's high-confidence output to the bin: samples with $p_i \approx 1$ have y in bin i (assuming a calibrated classifier), and since the bins

partition the y-axis in sorted order, the corresponding head outputs must also be sorted. At any other p, we would be making up a rule.

Formulation. For each head $i = 0, \dots, k-1$, define its high-confidence operating subset

$$S_i \;=\; \{x_n \,|\, p_i(x_n) \text{ in top decile of } \{p_i(x_1), \dots, p_i(x_N)\}\},$$

the training samples on which the classifier is most confident about class i. Define the **probability-weighted mean head output** on this subset:

$$M_i \;=\; \frac{\sum_{x \in S_i} p_i(x) \cdot h_i(p_i(x))}{\sum_{x \in S_i} p_i(x)}.$$

The weighting by $p_i(x)$ aligns M_i with the head's actual contribution to the LTV prediction ($\mu_{\text{total}} = \sum_j p_j\, h_j$): M_i is the average contribution of head i to μ_{total} on samples where head i dominates. Equivalently, it's the sensible *contribution* statistic — not just a function-value statistic.

(An unweighted arithmetic alternative $M_i^{\text{arith}} = |S_i|^{-1} \sum_{x \in S_i} h_i(p_i(x))$ yields the same ordering in any calibrated regime; the weighted form is strictly more sensitive to violations at the high-confidence end, where we most want sensitivity.)

Ordering penalty. Add to the training loss

$$\boxed{\mathcal{L}_{\text{order}} \;=\; \lambda \sum_{i=1}^{k-1} \big[\max(0,\; M_{i-1} - M_i + \delta)\big]^2}$$

where $\delta > 0$ is a small margin and λ is a weight. The total loss becomes $\mathcal{L} = \mathcal{L}_{\text{reg}} + \mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{order}}$ (with $\mathcal{L}_{\text{CE}}$ only active under `CE_STOP_GRAD` or the hybrid strategy).

The penalty is a hinge: zero when $M_i \geq M_{i-1} + \delta$, positive and smooth otherwise. The sum ranges over adjacent pairs, imposing strict ordering $M_0 < M_1 < \cdots < M_{k-1}$.

Identifiability proof sketch. Suppose we swap heads $i \leftrightarrow j$ with $i < j$. After the swap, M_i (now computed as the mean output of "the head formerly known as j" on the subset where p_i is highest) reflects what was previously M_j, which is higher. Similarly the new M_j reflects the old M_i. So $M_i^{\text{swap}} > M_j^{\text{swap}}$, violating the ordering for that pair. Hinge penalty fires. Any non-trivial permutation of heads creates at least one pair violating ordering, so the penalty is zero only at the identity permutation (and its discrete neighbourhood). Degeneracy broken.

Key properties.

- **No anchor at any specific p value.** The constraint uses the classifier's live probabilities to select subsets; no reference to $p = 1$ or any other fixed probability.
- **No classifier warmup needed.** M_i is computed from the current classifier's outputs at each training step. Under `CE_STOP_GRAD` the classifier stabilises quickly under bin-CE; under `REGRESSION_ONLY` the M_i co-evolve with the classifier, which is fine because the constraint only asks for ordering, not specific values.
- **Uniform across bins.** Every adjacent pair $(i-1, i)$ contributes one inequality; there is no special case for edges vs. middle bins.
- **Strictly weaker than all earlier proposals.** Does not bound the head's output, does not fix any head value at any p, does not constrain the head's functional form. Only a scalar ordering on the k means.

- **Minimal sufficient for identifiability.** The LTV loss has an S_k permutation symmetry; breaking it requires $k-1$ independent constraints (enough to pick out the canonical permutation). The ordering penalty provides exactly $k-1$.

Hyperparameters.

- δ: margin. A natural scale is $\delta \sim (y_{\max} - y_{\min})/k$ — one bin-width's worth of separation. Can also be set to a fraction of the residual scale, e.g. $0.1 \cdot \mathrm{std}(y)$.
- λ: weight of the ordering loss relative to regression loss. Start at 1.0 and tune. Too small and the constraint is ineffective; too large and the heads are pushed apart artificially.
- Decile cutoff for S_i: "top 10%" is a reasonable default; could be "top 5%" for tighter estimates. Robust to the exact choice.

Caveats.

- Computing M_i requires sorting each batch by p_i per head. For batch size B and k classes, this is $O(kB \log B)$ — negligible relative to the forward pass.
- When the classifier is still untrained (early epochs under `REGRESSION_ONLY`), top-decile subsets are nearly random and M_i is noisy; the ordering constraint will be weakly informative until the classifier converges. Not a failure mode — the ordering just becomes active once the classifier provides meaningful confidence.
- The constraint only acts in the high-p region. If the head misbehaves at moderate p values (e.g. outputs nonsensical values where $p_i \approx 0.3$), the ordering penalty is silent there. But this is fine: the regression loss already shapes the head on those samples through the LTV combination.

What the anchor does and does not do. The anchor is a constraint on the **head function** h_i at $p_i = 1$, not a constraint on the model output μ_{total} for samples in bin i. At $p_i = 1$ exactly, $\mu_{\text{total}} = h_i(1)$; but for $p_i < 1$ (which is almost always the case — empirically $\max_x p_i(x)$ lands in 0.8–0.95, not 1.0), the prediction μ_{total} is a convex combination of all heads and differs freely from $h_i(1)$. For samples near the lower boundary of bin i we *want* $\mu_{\text{total}} < c_i$ via $p_{i-1} > 0$ and h_{i-1} leaking the mean down; the anchor does not prevent this. The anchor is a symmetry-breaking *limit* condition, not a pointwise constraint on the prediction.

3.4 How the three are orthogonal

- MDN changes the likelihood.
- CE_STOP_GRAD changes the supervision of the classifier.
- Anchoring changes the head's parametrization.

All combinations are meaningful, giving the 8-cell matrix below. ClassReg serves as a fifth ground-truth-bin-labels baseline.

4. Experimental design

4.1 The 8-cell matrix

cell	loss	optimization strategy	anchored heads
A	Gaussian LTV	`REGRESSION_ONLY`	no
B	Gaussian LTV	`REGRESSION_ONLY`	yes

cell	loss	optimization strategy	anchored heads
C	Gaussian LTV	`CE_STOP_GRAD`	no
D	Gaussian LTV	`CE_STOP_GRAD`	yes
E	MDN	`REGRESSION_ONLY`	no
F	MDN	`REGRESSION_ONLY`	yes
G	MDN	`CE_STOP_GRAD`	no
H	MDN	`CE_STOP_GRAD`	yes

Cell A is the pre-existing default. Cell D pins all three dials at once.

4.2 Datasets (all $n = 600$–800, $d = 1$)

1. **heteroscedastic**: $y = 2\sin(x) + 0.5x + \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, (0.1 + 0.4|x|)^2)$, $x \sim \mathcal{U}(-5, 5)$. Smooth heteroscedastic noise; target range $\sim [-6, 6]$.
2. **bimodal**: $y = x + s \cdot 1.5 + \varepsilon$ with $s \sim \{-1, +1\}$, $\varepsilon \sim \mathcal{N}(0, 0.1^2)$, $x \sim \mathcal{U}(-3, 3)$. Two-mode posterior per input; tests the classification bottleneck.
3. **piecewise**: $y = 0.5x$ for $x < 0$, $y = 0.5x + \sin(4\pi x)$ for $x \geq 0$, plus Gaussian noise. Smooth-to-oscillatory transition; tests representational flexibility.
4. **exponential**: $y = e^x + \varepsilon$, $\varepsilon \sim \mathcal{N}(0, 0.5^2)$, $x \sim \mathcal{U}(-3, 3)$. Heavy-tail, all-positive target in $[\sim 0, \sim 20]$.

4.3 Grid

- $k \in \{3, 5\}$
- 3 seeds per configuration (42, 43, 44)
- 80 epochs, LR 0.01, early stopping at 15 epochs, 20% val split
- 9 cells (ClassReg + 8 ProbReg cells) $\times$ 4 datasets $\times$ 2 k $\times$ 3 seeds = 216 trainings

4.4 Metrics (measured on test split)

- MSE(μ_{total}, y) — point-prediction error.
- Gaussian NLL — uses predicted point + Gaussian uncertainty from `predict_uncertainty()`.
- **anchor_error** (ProbReg only): $\max_i |h_i(p_i^\star) - c_i|$ where $p_i^\star$ is the test point at which class i is most confident. Measures how close each head is to its bin centroid at its own argmax — a direct probe of head–bin alignment.
- **max_p_mid** (at $k = 5$): $\max_x p_{\lfloor k/2 \rfloor}(x)$ — whether the middle class is ever the argmax. Low values mean the middle class is "swallowed" by its neighbours.

5. Raw results — full 8-cell $\times$ 4-dataset $\times$ 2-k table

Means over 3 seeds. Rows sorted by dataset, k, cell. **Best MSE per (dataset, k) group is highlighted by eye.** Bug-induced runs have been rerun; all numbers here are from clean training.

dataset	k	cell	MSE	MSE std	NLL (Gauss)	anchor_err	max_p_mid
bimodal	3	A	2.372	0.040	1.853	3.672	—

dataset	k	cell	MSE	MSE std	NLL (Gauss)	anchor_err	max_p_mid
bimodal	3	B	2.397	0.079	1.858	0.030	—
bimodal	3	C	2.590	0.251	1.924	0.742	—
bimodal	3	D	2.589	0.212	1.921	0.110	—
bimodal	3	E	2.626	0.098	1.986	5.360	—
bimodal	3	**F**	**2.305**	0.039	1.851	1.801	—
bimodal	3	G	2.728	0.250	2.003	0.529	—
bimodal	3	H	2.599	0.193	1.941	0.071	—
bimodal	3	ClassReg	3.350	0.083	2.030	—	—
bimodal	5	**A**	**2.329**	0.050	1.844	3.804	0.370
bimodal	5	B	2.379	0.055	1.856	0.223	0.289
bimodal	5	C	2.542	0.087	1.911	0.720	0.746
bimodal	5	D	2.677	0.048	1.926	0.316	0.755
bimodal	5	E	2.722	0.052	2.081	4.592	0.167
bimodal	5	F	2.382	0.148	1.866	1.377	0.210
bimodal	5	G	2.712	0.123	2.007	0.355	0.781
bimodal	5	H	2.788	0.142	2.046	0.348	0.744
bimodal	5	ClassReg	3.393	0.092	2.032	—	—
heteroscedastic	3	A	2.190	0.387	1.647	3.985	—
heteroscedastic	3	B	1.956	0.245	**1.484**	0.059	—
heteroscedastic	3	**C**	**1.775**	0.060	1.651	0.599	—
heteroscedastic	3	D	1.958	0.128	1.672	0.108	—
heteroscedastic	3	E	2.413	0.158	1.754	3.580	—
heteroscedastic	3	F	2.194	0.717	1.602	0.071	—
heteroscedastic	3	G	1.834	0.156	1.694	0.299	—
heteroscedastic	3	H	1.971	0.084	1.745	0.089	—
heteroscedastic	3	ClassReg	2.013	0.189	1.769	—	—
heteroscedastic	5	A	2.124	0.202	1.586	3.371	0.355
heteroscedastic	5	**B**	**1.803**	0.141	**1.486**	0.739	0.958
heteroscedastic	5	C	1.998	0.203	1.669	0.354	0.594
heteroscedastic	5	D	1.872	0.165	1.646	0.269	0.588
heteroscedastic	5	E	2.080	0.050	1.684	3.280	0.847
heteroscedastic	5	F	2.102	0.119	1.612	0.971	0.988
heteroscedastic	5	G	1.860	0.131	1.650	0.426	0.594
heteroscedastic	5	H	1.963	0.255	1.680	0.207	0.553
heteroscedastic	5	ClassReg	2.076	0.118	1.786	—	—
piecewise	3	A	0.334	0.089	0.621	3.363	—
piecewise	3	**B**	**0.321**	0.047	0.665	0.018	—
piecewise	3	C	0.329	0.014	0.789	0.427	—
piecewise	3	D	0.414	0.119	0.826	0.070	—
piecewise	3	E	0.378	0.154	0.710	3.346	—
piecewise	3	F	0.332	0.043	0.717	0.061	—
piecewise	3	G	0.428	0.139	0.875	0.238	—
piecewise	3	H	0.351	0.053	0.825	0.056	—
piecewise	3	ClassReg	0.372	0.051	0.926	—	—
piecewise	5	**D**	**0.284**	0.020	0.631	0.186	0.893
piecewise	5	A	0.293	0.050	0.532	2.382	0.050
piecewise	5	E	0.292	0.048	0.573	2.496	0.728
piecewise	5	G	0.292	0.023	0.710	0.347	0.923
piecewise	5	B	0.298	0.050	0.599	0.148	0.669
piecewise	5	F	0.298	0.031	0.624	0.207	0.944
piecewise	5	H	0.298	0.054	0.633	0.085	0.946

dataset	k	cell	MSE	MSE std	NLL (Gauss)	anchor_err	max_p_mid
piecewise	5	C	0.311	0.030	0.624	0.352	0.927
piecewise	5	ClassReg	0.352	0.062	0.899	—	—
exponential	3	**E**	**0.507**	0.133	1.041	9.393	—
exponential	3	A	0.569	0.116	1.065	8.599	—
exponential	3	B	0.799	0.443	0.974	4.201	—
exponential	3	F	2.944	0.311	1.848	3.001	—
exponential	3	ClassReg	2.838	0.430	1.952	—	—
exponential	3	D	5.489	0.612	1.561	0.177	—
exponential	3	G	5.691	0.544	1.682	0.225	—
exponential	3	C	5.769	0.599	1.604	0.448	—
exponential	3	H	5.782	0.633	1.710	0.187	—
exponential	5	**A**	**0.503**	0.189	1.028	17.789	0.083
exponential	5	E	0.995	0.543	1.220	16.775	0.214
exponential	5	B	1.836	0.366	1.133	1.278	0.610
exponential	5	D	1.860	0.430	1.234	0.151	0.740
exponential	5	C	1.876	0.698	1.358	1.117	0.726
exponential	5	H	2.127	0.499	1.444	0.327	0.742
exponential	5	G	2.263	0.655	1.406	0.249	0.746
exponential	5	F	2.295	0.316	1.414	1.104	0.968
exponential	5	ClassReg	2.571	0.216	1.901	—	—

Raw CSV: `automl_package/examples/probreg_identifiability_results/summary.csv`. Per-seed rows: `results.csv` in the same directory.

6. Visual interpretation — what the probability-curve pages show

The per-dataset PDFs (`results_*.pdf`) contain two families of plots: `h_i(p_i)` (head output as a function of its own probability) and `p_i(x)` (classifier probabilities vs input). The visual signatures give intuition that the numbers alone can hide.

6.1 `h_i(p_i)` plots — head-index anchoring

What to look for: at $p_i = 1$, does head i's output land on the dotted horizontal line at centroid c_i?

- **Unanchored Gaussian cells (A, C):** heads drift arbitrarily — bright crossings, heads swapping ordering between seeds. On bimodal k=3 for A, head 0 climbs from -1 to $+2$ while head 2 falls from -0.5 to -3. These are two valid assignments; the loss does not disambiguate.
- **Unanchored MDN cells (E):** the worst head-swap cases; on exponential E shows anchor_error = 9.4 ($k = 3$) and 17.8 ($k = 5$). MDN's per-component likelihood *should* identify which component goes where, but with only $n = 600$ samples and wide-range centroids the model does not converge to the canonical assignment from random init.
- **Anchored cells (B, D, F, H):** every head's output terminates exactly at its own centroid as $p_i \to 1$. Curves are flat near their anchor and only deviate for small p_i, where the residual f_i has scope. anchor_error drops to ≤ 0.2 in most cases.
- **ClassReg:** heads are flat horizontal lines at the centroid values (heads are the output of a lookup table, not a learned NN). Included as reference.

6.2 `p_i(x)` plots — classifier signature

- **ClassReg:** always the sharpest. Classes form crisp, almost step-like partitions of x-space — the classifier has been trained on hard bin labels and commits to a decision boundary.
- **Cells G, H (MDN + CE_STOP_GRAD):** look nearly identical to ClassReg. This is the second visual confirmation of the identifiability claim: once the classifier is supervised by bin-CE, the probability curves become classifier-like regardless of the regression loss family.
- **Cells A, B (Gaussian + REGRESSION_ONLY):** softer, heavily overlapping curves. The regression loss tolerates diffuse p as long as μ_{total} is right — so the classifier never commits hard.
- **Cell E, F (MDN + REGRESSION_ONLY):** noisier, sometimes with one class collapsed (low $\max p$). MDN can trade off component weights freely, producing unstable curves on wider-range data.
- **Cells C, D on exponential (and only there):** classifier curves look classifier-like (as expected from CE_STOP_GRAD), but the regression heads behind them diverge from true bin means — see §8 for the mechanism.

6.3 What `max_p_mid` tells us at $k = 5$

Recall the middle class is $\lfloor k/2 \rfloor = 2$. `max_p_mid` is the maximum probability ever assigned to class 2 across test x. Low values mean the middle class is essentially unused.

- max_p_mid < 0.3: middle class is swallowed by its neighbours (seen in A and E, which lack any mechanism to keep it alive).
- max_p_mid $\geq$ 0.6: middle class actively used for some inputs. Anchored and CE_STOP_GRAD cells land here.

This is consistent with anchored heads and bin-CE supervision both providing structural pressure for every class to be meaningful.

7. Analysis and conclusions

7.1 Identifiability mechanism — ordering constraint (Gaussian-LTV + RegOnly only)

Anchored heads were the original candidate but were rejected on statistical grounds in §13 and replaced by the ordering-constraint penalty (§3.3, results in §9.1). The rerun shows:

- anchor_error drops 2–7× on Gaussian-LTV + REGRESSION_ONLY (cells A vs B), with neutral-to-improved MSE/NLL on 7/8 (dataset, k) cells.
- Under `CE_STOP_GRAD` the classifier's softmax weighting already pre-orders the heads (anchor ≤ 1.2 without the penalty), so the ordering term is a no-op — redundant.
- Combined with MDN it is harmful — mixture-parameter output does not share the monotone-means semantics, and exponential MSE degrades 3–4×.
- Margin must be $\delta = 0$; a range-normalised δ fails on heavy-tail geometries.

Recommendation: enable the ordering penalty only under `loss_type = gaussian_ltv` + `optimization_strategy = REGRESSION_ONLY`. Do not treat it as a universal default.

7.2 ClassReg vs ProbReg

ClassReg is consistently beaten by at least one ProbReg cell on all four datasets:

dataset	best ProbReg MSE	ClassReg MSE	gap
bimodal k=3	2.305 (F)	3.350	−31%
heteroscedastic k=3	1.775 (C)	2.013	−12%
piecewise k=3	0.321 (B)	0.372	−14%
exponential k=3	0.507 (E)	2.838	−82%

The gap is largest on bimodal and exponential, where a smooth posterior is critical. ClassReg's step-function classifier cannot interpolate between bins; ProbReg's soft probabilities can. On piecewise, where the target is nearly deterministic, the gap is small.

7.3 CE_STOP_GRAD — situational dial, not default

- Wins heteroscedastic MSE (Cell C = 1.775, beating the unanchored Gaussian A = 2.190 by 19%).
- Loses bimodal MSE (Cell C = 2.590 vs Cell A = 2.372, +9%).
- **Catastrophically fails on exponential** (MSE $\sim$ 5.5-5.8 across C, D, G, H vs ≤ 0.8 for A, B, E). See §8 for why.
- At $k = 5$, CE_STOP_GRAD cells scale worse — Cell C heteroscedastic goes 1.775 $\rightarrow$ 1.998 while Cell B goes 1.956 $\rightarrow$ 1.803.

CE_STOP_GRAD is best kept as an opt-in dial for bounded / symmetric targets where classifier semantics are wanted.

7.4 MDN NLL — situational, not uniform

- Wins exponential k=3 (Cell E = 0.507, best overall on that row).
- Wins bimodal k=3 (Cell F = 2.305, beats A by $\sim$ 3%).
- Loses on heteroscedastic and piecewise (Gaussian LTV wins both).
- MDN + ordering is **not** a safe combination — ordering head means fights MDN's free mixture-parameter assignment (see §9.1, E vs F).

Keep Gaussian LTV as the default loss; expose MDN as a `prob_reg_loss_type` dial for bimodal / heavy-tail data, and disable the ordering penalty when MDN is selected.

7.5 Recommendation matrix

setting	default	expose as dial	rationale
ordering constraint	**on for Gauss-LTV + RegOnly only**		2–7× anchor drop; redundant under CE_STOP_GRAD, harmful under MDN

setting	default	expose as dial	rationale
Gaussian LTV vs MDN	Gaussian	yes	MDN is situationally better, not uniformly
`REGRESSION_ONLY` vs `CE_STOP_GRAD`	`REGRESSION_ONLY`	yes	CE_STOP_GRAD breaks on heavy-tail targets
ClassReg vs ProbReg	**ProbReg**		ClassReg only competitive on narrow-range targets

7.6 The best "safe default" overall

Cell B — Gaussian LTV + `REGRESSION_ONLY` + ordering penalty. It either wins or ties-best on NLL across all four datasets, gets a 2–7× anchor reduction over Cell A, and is the highest among cells that do not fail catastrophically on any. If classifier semantics are specifically wanted (e.g. calibration / interpretability applications where we need $p_i(x)$ to mean "probability of bin i"), **Cell C** (no ordering needed — CE_STOP_GRAD pre-orders for free) is the right choice at the cost of mild MSE degradation outside exponential.

7.7 Default policy (codified in `ProbabilisticRegressionModel`)

The three cells with simultaneously (i) solved identifiability and (ii) no catastrophic regression across datasets are **B, C, G**. The underlying principle: something must break the S_k head-index swap symmetry — either the explicit ordering penalty (B) or the implicit pinning provided by a frozen classifier (C, G). Combining both is redundant at best (D, H) and harmful with MDN (F). Leaving both off is unidentified (A, E).

Policy implemented via auto-resolution of `ordering_constraint_weight` in `ProbabilisticRegressionModel.__init__`:

user passes	loss	opt strategy	strategy	resolved weight
`None` (default)	Gaussian-LTV	`REGRESSION_ONLY`	`SEPARATE_HEADS`	**1.0** (Cell B)
`None` (default)	any other combination			**0.0**
explicit float	—	—	—	**used verbatim**

Guardrails:

- Explicit `ordering_constraint_weight > 0` with `prob_reg_loss_type = MDN` emits a warning pointing to §9.1 (cells E vs F regression on exponential). We do not silently override — research users may want to verify the finding.
- Non-`SEPARATE_HEADS` strategies silently no-op the penalty in `_calculate_custom_loss` regardless of the weight (the per-head output tensor needed for M_i doesn't exist in other strategies).

Opt-in dials users should consider overriding:

- Heteroscedastic-style noise structure: switch to **Cell C** (`optimization_strategy = CE_STOP_GRAD`). Wins MSE by 10–17% on heteroscedastic data.
- Multimodal / heavy-tail targets where calibration matters: **Cell G** (`prob_reg_loss_type = MDN` + `optimization_strategy = CE_STOP_GRAD`). Ordering auto-resolves to 0 for this combination.

Never-recommended configurations: A, D, E, F, H. The auto-resolution prevents the common accidental A (unidentified default) but does not prevent D/E/F/H if explicitly chosen — those remain available for ablations and research.

7.7.1 Loss-term typology: supervision vs regularization

There is a philosophical tension between Cell B (ordering) and Cell C (CE_STOP_GRAD) that the empirical ranking does not resolve:

- **(i) Supervision on a separate target.** CE_STOP_GRAD adds a cross-entropy loss $\mathcal{L}_{\text{CE}}(p_\cdot(x), b(y))$ on percentile-bin labels $b(y)$. The bin label is a deterministic function of y and percentile cut-points, so $p_i(x)$ is being supervised to approximate $\Pr[b(y) = i \mid x]$ — a well-defined probabilistic quantity. The regression loss (Gaussian-LTV NLL) is itself probabilistically motivated, so layering a likelihood on bin labels is consistent with the generative story.
- **(ii) Regularization on the regression path.** The ordering penalty $\mathcal{L}_{\text{order}}$ operates on M_i, a probability-weighted mean of $h_i(p_i)$ over the top-decile-p_i subset. It pushes the regression heads directly, and is not the gradient of any likelihood — it is a constraint chosen to break the S_k swap symmetry.

On grounds of probabilistic cleanliness, (i) > (ii): once we accept regularization terms of type (ii), there is no principled stopping rule. Would we add a middle-class occupancy penalty too? A head-spread penalty? Each one is individually defensible yet the coefficient space grows without end.

Despite this, **B remains the default** on empirical grounds:

- B wins or ties-best on NLL against C on 5/6 non-exponential dataset–k cells.
- C fails catastrophically on heavy-tail targets (exponential $k = 3$, C MSE = 5.77 vs B MSE = 0.51 — a $\sim 11\times$ regression).
- C's middle-bin probability under $k = 5$ runs to $\max p_{\text{mid}} \in [0.59, 0.93]$ — CE forces bin occupancy whether or not the data support k effective components, which can mask effective-$k <$ nominal-k collapses that B surfaces honestly (see §7.8).

The tension is real and worth revisiting after the k-sweep of §9: if C + dynamic-k closes the exponential gap and keeps the NLL parity on the others, the philosophical argument becomes actionable.

7.8 Against middle-class penalty stacking

A tempting follow-up to the ordering constraint is a **middle-class occupancy penalty**: for $k \geq 5$, the unidentified Cell A collapses the middle bin to $\max p_{\text{mid}} \approx 0.05$–$0.37$, and Cell B only partially recovers ($[0.31, 0.54]$). One could add a penalty $\mathcal{L}_{\text{mid}} = -\lambda_{\text{mid}} \sum_{i \in \text{mid}} \log \bar{p}_i$ (or similar) that pushes the batch-averaged middle-bin mass toward $1/k$.

We explicitly decline to do this, for two reasons:

1. **Slippery slope on type-(ii) regularizers.** Once the coefficient vector contains one hand-chosen penalty on the regression path, the threshold for adding more collapses: head-spread, variance-floor, anti-collapse, anchor-drift… each individually defensible, none grounded in a likelihood. See §7.7.1: type-(i) supervision has a natural stopping rule (supervise observables); type-(ii) regularization does not. We draw the line at one (the ordering penalty, whose identifiability role is unreplaceable in cells B/F).
2. **Emptiness is information, not a bug.** A middle bin with $p_i \to 0$ at large k is the model telling us the effective number of components supported by the data is smaller than the nominal k we passed in. Stamping that out with a penalty hides the signal. The correct response is to let k adapt to the data — which is exactly what the dynamic-k machinery (`NClassesSelectionMethod` $\in$ {Gumbel, SoftGating, STE, REINFORCE} combined with `NClassesRegularization` $\in$ { K_PENALTY, ELBO}) is for.

The test of (2) is §9 (P2.3): if dynamic-k + Cell B recovers best-fixed-k performance or better, the middle-class emptiness problem is absorbed into the k-selection mechanism and no penalty is needed. If dynamic-k fails to improve over fixed k, we revisit — but through the lens of fixing the selection mechanism, not by stacking another regularizer.

Corresponding work — a middle-class centering penalty — is struck from the roadmap.

8. The symlog hypothesis — a negative result

8.1 The diagnosis we proposed

On exponential ($y \in [\sim 0.05, \sim 20]$), percentile bins put $k = 3$ centroids roughly at $c = (0.2, 1.5, 12)$. Their spread is enormous. Evaluating the LTV between-class variance at uniform $p = (1/3, 1/3, 1/3)$ and with head outputs initialised near their centroids:

$$\underbrace{\tfrac{1}{3}(0.04 + 2.25 + 144)}_{\approx 48.8} \; - \; \underbrace{(\tfrac{1}{3}(0.2 + 1.5 + 12))^2}_{\approx 20.9} \; \approx \; 27.9.$$

So $\sigma^2_{\text{total}} \gtrsim 28$ at initialisation. The Gaussian NLL gradient with respect to any head's mean at a sample y is

$$\frac{\partial \mathcal{L}_{\text{LTV}}}{\partial \mu_j} \; \propto \; -\frac{p_j \, (y - \mu_{\text{total}})}{\sigma^2_{\text{total}}}.$$

With $\sigma^2_{\text{total}} \approx 28$, a residual of magnitude ~ 3 produces gradient contributions $\sim 0.1 \cdot p_j$ — an order of magnitude smaller than on bimodal ($\sigma^2_{\text{total}} \approx 2$). So heads receive weak signal.

Under `REGRESSION_ONLY`, the regression loss can still adapt the classifier to collapse p toward a single class, which reduces σ^2_{total} quickly and unblocks the head gradient. Under `CE_STOP_GRAD`, the classifier is pinned by bin-CE and cannot compress p; the heads stay stuck on this "flat loss plateau" through training. This would explain the 10× MSE catastrophe on exponential for C/D/G/H.

Predicted fix: `target_transform="symlog"` compresses y to $\text{symlog}(y) = \text{sign}(y) \log(1 + |y|) \in [\sim 0.05, \sim 3.04]$. In that space, centroids are roughly $(0.18, 0.92, 2.56)$ and the between- class variance drops to ~1.1 — comparable to bimodal. Predicted outcome: heads recover proper gradient and C/D converge.

8.2 The experiment

Script: `automl_package/examples/exponential_symlog_probe.py`. 96 trainings = 8 cells × 2 k × 2 transforms × 3 seeds on exponential. Results at `automl_package/examples/exponential_symlog_probe_res`

8.3 Results

MSE, mean over 3 seeds:

cell	k	none	symlog	verdict
A	3	0.69	0.79	baseline, unaffected
B	3	1.91	2.05	slight regression
C	3	**4.43**	**4.49**	**no improvement**
D	3	**4.08**	**4.12**	**no improvement**
E	3	0.50	1.67	symlog *hurts* unanchored MDN
F	3	1.85	**0.40**	**5× improvement** (MDN + anchored)
G	3	4.17	4.21	~same
H	3	4.70	4.07	marginal
A	5	0.40	0.47	baseline
C	5	2.78	2.67	marginal
D	5	1.90	2.06	slight regression
F	5	1.92	1.23	moderate improvement

8.4 What it rules out and what it leaves

The target-range-compression story is **not** the dominant cause. C/D stay broken on exponential under symlog by margins indistinguishable from the original training. The inter-head variance term is real math, but either (a) it is not the binding constraint in practice, or (b) the classifier under CE_STOP_GRAD is itself not learning well enough on exponential *regardless* of target scale.

Surprise finding: symlog rescues Cell F (MDN + anchored) by 5× on exponential k=3. This is not what we predicted to improve. Our best explanation: MDN per-component likelihoods are sensitive to target scale (the $\log \sigma_i$ term grows with range), and symlog removes that pressure. Anchored heads absorb the residual head-swap risk MDN would otherwise bring back. This combo is interesting for heavy-tail real applications (photo-z, cluster mass).

8.5 Remaining suspects for the C/D failure

1. **Percentile bins on heavy-tail** y: bin $k-1$ on exponential spans $[3, 20]$ — a single mean per bin is a terrible fit. Under REGRESSION_ONLY, regression loss can compress p to shift mass toward specific samples within the bin and still fit them tolerably (because μ_{total} is a mixture). Under CE_STOP_GRAD, each head must commit to a single bin-mean value and is punished everywhere else in the bin. This would explain why larger k helps (C $k = 5 = 2.78$ vs $k = 3 = 4.43$ — halves with finer bins).
2. **No classifier/head co-adaptation** under CE_STOP_GRAD means the training regime is a two-player non-cooperative game rather than a joint-optimization — heads get worse gradient statistics.

3. **Head initialization**: anchored heads start at centroids (near correct), unanchored heads at 0 (far from the 12 centroid). Yet anchored cells D/H still fail — ruling out "bad init" as the primary cause and re-pointing at bin/target geometry.
4. **Anchor target mis-specification** (see §3.3): pinning $h_i(1) = c_i$ is wrong for edge bins because $\mathbb{E}[y \mid p_i(x) = 1] \neq c_i$ when the bin is asymmetric about its mean. On exponential's upper bin the correct value is well above c_{k-1}, so the current anchor forces a systematic under-prediction on the upper tail. The proposed fix is the **ordering-constraint** soft loss described in §3.3: drop the hard anchor entirely and let the heads run unconstrained, adding only a hinge penalty on the *ordering* of their high-confidence operating means $M_0 < M_1 < \cdots < M_{k-1}$. This is the most likely root cause of the anchored cells (B, D, F, H) still failing on exponential and is a concrete actionable fix — worth trying *before* the hybrid opt-strategy (now §9.2).

9. Open questions and next experiments

In priority order.

9.1 Ordering constraint — results from the rerun

The current anchored head (which imposes $h_i(1) = c_i$ hard) was replaced with an **ordering soft penalty** from §3.3. No change to head parametrization: heads remain plain `BaseRegressionHead` with full flexibility. Added loss term:

$$\mathcal{L}_{\text{order}} \;=\; \lambda \sum_{i=1}^{k-1} \big[\max(0,\; M_{i-1} - M_i + \delta)\big]^2$$

where

$$M_i \;=\; \frac{\sum_{x \in S_i} p_i(x)\, h_i(p_i(x))}{\sum_{x \in S_i} p_i(x)}, \qquad S_i = \{x : p_i(x) \text{ in top } 10\%\}$$

is the probability-weighted mean of head i over the top-decile-p_i subset of the batch.

Hyperparameters used in the sweep. $\lambda = 1.0$, $\delta = 0$. An initial attempt with $\delta = (y_{\max} - y_{\min})/k$ (one bin-width margin) catastrophically hurt exponential (MSE $\gg 5$) because the target-quantile centroids on a heavy tail are far from uniformly spaced — margin-0 is the only safe default across problems with varying tail geometry.

Implementation: `automl_package/utils/ordering_loss.py` + `_calculate_custom_loss` integration in `automl_package/models/probabilistic_regression.py`. Under `CE_STOP_GRAD` the classifier logits are detached before entering the ordering term so the gradient-stop contract is preserved.

Validation: re-ran the 8-cell matrix with anchored heads disabled and ordering toggled on/off. Three seeds per cell, $k \in \{3, 5\}$, four datasets. Clean run against post-fix code; the gradient-stop fix itself moves D/H by less than seed noise (max $|\Delta\text{MSE}| = 0.05$).

9.1.1 Measured ordering-on vs ordering-off pairs

Ratio $r_{\text{anchor}} = \text{anchor\_on/anchor\_off}$ (lower = stronger symmetry break). MSE columns in data units.

Cells A vs B — Gaussian-LTV + REGRESSION_ONLY (ordering's target use case):

dataset	k	MSE off	MSE on	r_{anchor}
bimodal	3	2.372	2.349	0.148
bimodal	5	2.329	2.386	0.372
exponential	3	0.569	0.509	0.897
exponential	5	0.503	1.696	0.178
heteroscedastic	3	2.190	2.142	0.174
heteroscedastic	5	2.124	2.206	0.536
piecewise	3	0.334	0.292	0.229
piecewise	5	0.293	0.296	0.503

Seven of eight cells: anchor drops 2–7× with neutral-to-improved MSE/NLL. One regression: exponential $k = 5$ (MSE $0.50 \to 1.70$) — the heavy-tail geometry is hostile to uniform-weight ordering even at $\delta = 0$.

Cells C vs D — Gaussian-LTV + CE_STOP_GRAD: $r_{\text{anchor}} \in [0.90, 1.15]$; $|\Delta\text{MSE}| \leq 0.22$ (all within seed std). Ordering is **redundant** here because the classifier's softmax weighting already pre-orders the heads — anchor error is already ≤ 1.2 without the penalty.

Cells E vs F — MDN + REGRESSION_ONLY: anchor drops as with A/B, but MSE **regresses on exponential** ($0.51 \to 1.91$ at k=3; $0.99 \to 3.17$ at k=5) and MDN NLL worsens on bimodal k=3 ($0.18 \to 0.49$). The head-means ordering is the wrong semantics for MDN's mixture-parameter output — its "heads" predict $(\mu_i, \log \sigma_i^2, \pi_i)$ triples, and forcing μ_i monotone in i conflicts with the free mixture assignment.

Cells G vs H — MDN + CE_STOP_GRAD: essentially identical ($|\Delta\text{MSE}| \leq 0.07$), same reason as C vs D.

9.1.2 Verdict

The ordering penalty **achieves its identifiability goal** (5–7× anchor reduction) **only for Gaussian-LTV + REGRESSION_ONLY**. Elsewhere it is either redundant (CE_STOP_GRAD variants) or actively harmful (MDN + REGRESSION_ONLY).

Updated defaults:

- `use_ordering_constraint = True` **only** when `loss_type = gaussian_ltv` and `optimization_strategy = REGRESSION_ONLY`.
- Leave off under CE_STOP_GRAD (no benefit, slight compute cost).
- **Do not combine** with MDN.
- `margin = 0.0` unconditionally — the range-normalised margin hypothesised in earlier drafts fails on heavy-tail targets.

Outstanding: exponential $k = 5$ under the recommended config still regresses relative to ordering-off. Worth a small tuning pass ($\lambda \in \{0.1, 0.3\}$) or switching the exponential pipeline to CE_STOP_GRAD (where ordering is a no-op anyway).

Measured data: `automl_package/examples/probreg_identifiability_results/summary.csv` (clean), `summary.TAINTED-DH.csv` (pre-fix baseline used to verify the gradient-stop leak had no observable effect).

9.2 Hybrid opt-strategy

Train jointly (`REGRESSION_ONLY`) for the first ~30 epochs, then switch to `CE_STOP_GRAD` for the remainder. Gives classifier + heads time to co-adapt before locking the classifier. Would validate or refute suspect (2) in §8.5.

Implementation sketch: add `ProbabilisticRegressionOptimizationStrategy.HYBRID` with a `ce_stop_grad_after_epoch: int = 30` knob in `probabilistic_regression.py`. At inference-time the heads see detached probs; the classifier stays frozen on bin-CE post-switch.

Train jointly (`REGRESSION_ONLY`) for the first ~30 epochs, then switch to `CE_STOP_GRAD` for the remainder. Gives classifier + heads time to co-adapt before locking the classifier. Would validate or refute suspect (2) in §8.5.

Implementation sketch: add `ProbabilisticRegressionOptimizationStrategy.HYBRID` with a `ce_stop_grad_after_epoch: int = 30` knob in `probabilistic_regression.py`. At inference-time the heads see detached probs; the classifier stays frozen on bin-CE post-switch.

9.2 Adaptive bin boundaries on heavy-tail y

Replace percentile bins (which widen at the heavy tail) with uniform-on- symlog-y bins on wide-range targets. Would test suspect (1).

9.3 Per-head log-variance warm start

Initialise each head's $\log \sigma_i^2$ bias from $\log \mathrm{Var}[y \mid y \in \mathrm{bin}_i]$. Makes the NLL well-scaled from step 1 even when the classifier is pinned. Cheapest intervention; should test before (9.1).

9.4 Anchored + MDN + symlog on real domains

The Cell F + symlog combo was the most dramatic improvement we've seen. Test it on photo-z (targets span orders of magnitude in galaxy redshift) and cluster mass (log-normal mass function). If it generalises, it's a candidate Paper A configuration specifically for heavy-tail astrophysics.

9.5 Re-test Cell D with classifier warm-start

Train the classifier alone on bin-CE for 10 epochs (no regression loss active), then flip regression loss on with `CE_STOP_GRAD`. Should reveal whether the classifier is the bottleneck or the heads are.

9.6 Larger k sweep on exponential

Already partial evidence that $k = 5$ halves C's MSE from $k = 3$. Run $k \in \{3, 5, 10, 20\}$ on exponential to confirm the trend and find the turn-over where intra-bin variance is negligible.

10. Bug log (during this investigation)

Four bugs were found and fixed (three new, one documented-but-unfixed) while running this sweep. Durable record here so future runs don't repeat them.

10.1 SINGLE_HEAD_FINAL_OUTPUT + dynamic-k shape mismatch (280ad1f)

`_compute_predictions_for_k` had a special branch for `SINGLE_HEAD_FINAL_OUTPUT` that applied a weighted sum over per-class outputs, but that strategy already returns `(batch, output_size)` — the weighted sum caused a shape mismatch. Crashed all dynamic-k ProbReg runs with this strategy.

Affected: 12/48 configs of `probreg_ablation` (first overnight run). Reran after fix; clean now.

10.2 `calculate_class_value_ranges` numpy.float32 rejection (19bdacc)

PyTorch 2.6+ refuses to assign `numpy.float32` scalars to tensor elements via `tensor[i, j] = scalar`. Caller passes `np.min(y)` which returns a `numpy.float32` when y is `float32`.

Affected: all 50 `ProbReg+HPO` trials on UCI-Yacht in the original HPO sweep (produced NaN across the board because boundary regularisation was in the Optuna search space). Rerun after fix.

10.3 BatchNorm with batch size 1 (fe63c90)

UCI-Yacht has 308 samples; after the 0.3 test split and internal 0.1 test / 0.2 val splits, the ProbReg HPO final training saw $n_{\text{train\_val}} = 193$ samples. With `batch_size=32`, $193 \bmod 32 = 1$ — the last batch contains a single sample, and `BatchNorm` in training mode refuses batches of size 1.

Fix: `drop_last=True` when the last batch would have exactly 1 sample. Triggered only under HPO on UCI-Yacht; other sweeps use datasets whose sizes do not produce a 1-sample remainder.

10.4 `final_results_report.py` column name (fe63c90)

The aggregator expected a column `nll_mean` but the identifiability sweep CSV produces `nll_own_mean`, `nll_gaussian_mean`, `nll_mdn_mean`. Fixed to tolerate either.

10.5 B1 — monotonic head init with `mean=-3.0` (pre-existing, not fixed)

`_initialize_monotonic_head` uses `nn.init.normal_(weight, mean=-3.0)`, which breaks on all-positive targets. **Does not affect this investigation** — the identifiability sweep sets `use_monotonic_constraints=False`. Affects `head_degeneracy_diagnostic` and `overfitting_showcase` on exponential; their exponential results should be disregarded until B1 is fixed.

11. File, artefact, and commit pointers

Scripts

purpose	path
main sweep	`automl_package/examples/probreg_identifiability_sweep`
symlog probe	`automl_package/examples/exponential_symlog_probe.py`
aggregator PDF	`automl_package/examples/final_results_report.py`
implementation plan	`docs/probreg_identifiability_implementation_plan.md`

purpose	path

Result CSVs

purpose	path
per-seed results	`automl_package/examples/probreg_identifiability_resul`
cross-seed summary	`automl_package/examples/probreg_identifiability_resul`
symlog probe	`automl_package/examples/exponential_symlog_probe_resu`

PDFs (all under `automl_package/examples/probreg_identifiability_results/`)

- `results_bimodal.pdf` — metrics table + head curves + probability curves
- `results_heteroscedastic.pdf`
- `results_piecewise.pdf`
- `results_exponential.pdf`

Merged master PDF: `automl_package/examples/final_results_report/final_report.pdf` (concatenates the sweep-level summary with every per-sweep PDF; 38 pages via `pdfunite`).

Key commits

- `280ad1f` — fix: SINGLE_HEAD_FINAL_OUTPUT dynamic-k shape mismatch
- `19bdacc` — fix: numpy.float32 in calculate_class_value_ranges
- `fe63c90` — fix: BatchNorm batch_size=1 + report column name
- `4d7d9fe` — final_results_report: merge per-sweep PDFs via pdfunite
- `7b92ead` — restore GRADIENT_STOP semantics, RegressionHead ABC

12. What we'd pick up first in a future session

1. Implement the **ordering-constraint penalty** (§9.1). Drop the hard anchor in `AnchoredHead`, add $\mathcal{L}_{\text{order}}$ to the loss, re-run the 8-cell identifiability sweep. Expected payoff: edge-bin cells (B, D, F, H) on exponential should recover toward the unanchored baseline values.
2. Implement the **hybrid opt-strategy** (§9.2). Most likely source of insight about whether the CE_STOP_GRAD failure on exponential is co-adaptation or something else.
3. Run the **MDN + symlog** combination on photo-z / cluster-mass real data (§9.4) — one dramatic positive surprise on exponential and the combination most likely to matter for Paper A.
4. Consider the redesign choices flagged in §13 (MDN as default, adaptive bins, $h_i(x)$ instead of $h_i(p_i)$) before committing the Paper A architecture.

13. Statistical critique — does this model make sense?

The preceding sections focused on specific technical problems (head-swap degeneracy, anchor target, CE_STOP_GRAD failures) and concrete fixes (anchored heads, ordering constraint, symlog). This section steps back and asks whether the underlying statistical model is sound — independent of how we patch the surface symptoms.

13.1 The implied data-generating model

ProbReg in `SEPARATE_HEADS` mode implicitly assumes

$$y \,|\, x \;\sim\; \sum_{j=1}^{k} p_j(x)\, \mathcal{N}(\mu_j(p_j(x)),\; \sigma_j^2(p_j(x))),$$

a mixture of k Gaussians whose mixing weights are functions of x and whose component parameters are functions of the mixing weights themselves (not of x directly — this is a real restriction; see §13.4).

This is a well-defined statistical model. The MDN likelihood is the proper scoring rule for it.

13.2 Gaussian-LTV is not a proper mixture likelihood

The alternative Gaussian-LTV training objective

$$\mathcal{L}_{\text{LTV}} = -\tfrac{1}{2}\left[\log(2\pi\sigma^2_{\text{total}}) + \frac{(y-\mu_{\text{total}})^2}{\sigma^2_{\text{total}}}\right]$$

computes the log-density of a **moment-matched Gaussian approximation** to the mixture, not the mixture's own log-density. $(\mu_{\text{total}}, \sigma^2_{\text{total}})$ are correctly derived via LTV; the likelihood is a valid proper scoring rule for the Gaussian family, so it gives consistent estimates of $(\mu_{\text{total}}(x), \sigma^2_{\text{total}}(x))$. But it *cannot distinguish* different mixtures sharing these two moments. Multimodality is invisible to it.

Practical implications:

1. **Identifiability.** $\mathcal{L}_{\text{LTV}}$ identifies the mixture only up to its first two moments. The individual components (p_j, μ_j, σ_j^2) are not identifiable from the likelihood alone. This is the head-swap degeneracy restated in statistical language. All the anchoring and ordering machinery in §3.3 is a statistical remedy for a likelihood that leaves components under-determined.
2. **Proper scoring.** MDN is proper; Gaussian-LTV is proper for a *different* parametric family. Training under Gaussian-LTV does not give the MLE of the mixture model.
3. **Component interpretation.** Under Gaussian-LTV, the components (μ_j, σ_j^2) have no intrinsic meaning beyond "they aggregate to the moments we care about". Under MDN, the components are identifiable (up to permutation) and interpretable as actual mixture components with real marginal meaning.

Verdict: Gaussian-LTV is defensible as a computational simplification when you only want $\mu_{\text{total}}, \sigma^2_{\text{total}}$, and you don't need the components for anything downstream. If the components matter (e.g. for calibration, interpretability, or tighter uncertainty quantification on multimodal targets), MDN is the statistically coherent choice.

13.3 The classification bottleneck

The model compresses x to a k-dimensional softmax vector $p(x)$ before reconstructing y. This is an **information bottleneck** at dimension k.

Sufficiency question. Is $p(x)$ a sufficient statistic for y given x? In general, no — $p(x)$ is a lossy summary. A standard regressor on x has access to everything $p(x)$ loses.

Bias–variance trade-off. Compressing x through a classifier is a form of regularisation: it introduces bias (we cannot in general recover $E[y|x]$ exactly) in exchange for reduced variance (the classifier is more stable than a regressor under noisy supervision, because sign matters more than magnitude in classification loss). This is sensible when:

- Noise in y is large relative to within-bin structure.
- The classifier's bin assignment captures the dominant signal in $E[y|x]$.
- Within-bin residual structure is small or uninformative.

It is not sensible when within-bin structure is large and captures important features of $E[y|x]$ — the bottleneck then destroys useful signal.

Empirically this has been validated partially (§2 of RESUME.md, noise-robustness benchmark): at $\sigma = 1.0$, ClassReg with $k = 2$ beats tree baselines on smooth data. At low noise, standard regressors beat ClassReg. The classification bottleneck earns its keep specifically in the high-noise regime that motivated it.

13.4 $h_i(p_i)$ — strong conditional-independence assumption

The heads take $p_i(x)$ as input, not x. This encodes the assumption

$$E[y|x, y \in \text{bin}_i] \;=\; E[y|p_i(x), y \in \text{bin}_i],$$

i.e. $p_i(x)$ is a sufficient statistic for y given bin i. All samples x with the same $p_i(x)$ get the same μ_i, σ_i^2.

This is **stricter than standard MDN**, where μ_j, σ_j^2 depend on x directly. The ProbReg restriction reduces head capacity and makes the model rely more heavily on the classifier to carry position information.

On 1-D toy data, $p_i(x)$ is a smooth function of x and the restriction is mild (the head can reach most within-bin structure via p_i). On higher-dimensional inputs, different x's can give the same p_i with very different residual structure, and the restriction will bite. This is one of the architectural knobs worth revisiting before Paper A: allowing $h_i(x)$ (or $h_i(x, p_i)$) gives strictly more capacity.

13.5 Percentile bins + Gaussian component — model misspecification

Percentile bins give each bin equal *probability mass* but unequal *widths*. On heavy-tailed data, the top bin is wide and contains a skewed within-bin distribution. A single Gaussian $\mathcal{N}(\mu_i, \sigma_i^2)$ cannot represent a skewed conditional.

The within-component assumption

$$y \,|\, \text{bin}_i \;\approx\; \mathcal{N}(\mu_i, \sigma_i^2)$$

is reasonable only when the within-bin distribution is (approximately) symmetric and unimodal. For heavy-tailed or sharply skewed targets, it is misspecified. The anchor-target mis-specification we identified in §3.3 is a surface symptom: on a skewed bin the conditional mean c_i sits far from both the conditional mode and the deep-interior samples, and no point anchor at c_i can rescue the Gaussian fit.

This is model misspecification, not identifiability. No reparametrisation, no ordering constraint, no anchoring trick rescues a Gaussian on a skewed conditional distribution. The fix has to be either:

- **Finer bins** so within-bin skew is reduced (see §9.6).
- **Adaptive bins** on transformed y (e.g. uniform-on-symlog-y).
- **Target transforms** so the conditional becomes approximately Gaussian in transformed space.
- **Heavier-tailed component distributions** (Student-t, log-Normal) instead of Gaussian — a larger architectural change.

13.6 The ordering constraint, in statistical language

Under Gaussian-LTV the likelihood leaves components unidentified; the ordering constraint supplies $k-1$ inequalities $M_0 < M_1 < \cdots < M_{k-1}$ that pick a canonical labelling out of the $k!$ permutations. This is standard practice in Bayesian mixture fitting ("sorted-means" labelling convention) and recovers identifiability up to the minimal symmetry the data can support.

Under MDN the likelihood already identifies components up to permutation; ordering is redundant but harmless — it selects a canonical permutation for free.

In both cases, the ordering penalty is the statistical equivalent of a sorted-means prior. It is statistically standard and unproblematic.

13.7 Summary verdict

Aspect	**Statistical soundness**
MDN likelihood on mixture	**Sound.** Proper scoring rule; identifiable up to permutation.
Gaussian-LTV likelihood on mixture	**Partially sound.** Proper Gaussian scoring rule, but misspecifies the mixture; components unidentified.
Ordering constraint	**Sound.** Standard labelling convention; equivalent to sorted-means prior.
Classification-only bottleneck on x	**Sound as regularisation.** Loses information; trade-off depends on noise level.
h_i as function of p_i only (not x)	**Strong assumption.** Conditional-independence restriction; likely binding on high-d real data.
Percentile bins + Gaussian component	**Misspecified for heavy-tail data.** No identifiability fix compensates.
Overall model	Coherent for unimodal/mildly-multimodal, noisy, low-d regression; misspecified for heavy-tail or high-d non-compressible targets.

13.8 If redesigning from scratch

With the benefit of everything we've learned, a statistically cleaner architecture would:

1. **Use MDN as the primary likelihood.** Proper scoring rule, identifiable up to permutation, interpretable components. Gaussian-LTV becomes a secondary / diagnostic objective, not the default.
2. **Use adaptive bins** — uniform-on-symlog-y or uniform-on-transformed-y — to reduce within-bin skew on heavy-tail targets. Or abandon fixed binning and let the mixture components be

data-driven.

3. **Let heads depend on** x, not just p_i. The ProbReg restriction is a capacity bottleneck that is under-motivated for high-d data.
4. **Keep the ordering constraint** as the identifiability convention. It's statistically standard and costs nothing.
5. **Keep the classification bottleneck interpretation** as a valuable regularisation mode for noisy data — but treat it as *one setting of* k, not as the whole model. With k very large, the model converges to a mixture density network; with k very small, to a classification bottleneck. Both are useful operating points.

Whether the resulting model is still called "ProbReg" is a naming question. The core statistical insight — "combine classification robustness with regression precision via a mixture model" — survives the redesign; the specific parametrisation choices that cause our current pain points do not.